

RAPPORT D'AUDIT

Application CRM LiVrai

Analyse fonctionnelle, UX et technique de l'application existante

Mars 2026 - Laurent Gourouvin

Table des matières

1. Contexte et périmètre	3
2. Vue fonctionnelle	3
2.1 Parcours Admin.....	3
2.2 Parcours Client.....	4
2.3 Diagramme de cas d'utilisation	5
2.4 Diagramme d'activité.....	6
3. Expérience utilisateur.....	6
3.1 Anomalies d'affichage	6
3.2 Points de friction identifiés.....	7
4. Description technique	7
4.1 Architecture générale	7
4.2 Authentification.....	7
4.3 Couche DAO	7
4.4 Beans	8
4.5 Servlets	8
4.6 JSP (Views).....	8
4.7 Diagramme de composants	9
5. Points forts et déficiences	9
5.1 Points forts	9
5.2 Déficiences et risques	9
6. Conclusion	10

1. Contexte et périmètre

LiVrai est une entreprise de livraison B2B en forte croissance, aussi bien par le nombre de clients que par le volume de commandes. La société s'appuie sur une application CRM pour gérer ses relations clients et ses livraisons.

Face à cette croissance, les équipes constatent des lenteurs, des difficultés de maintenance et s'interrogent sur la disponibilité du service lors des pics d'activité. Cette pression directe sur le CRM rend un état des lieux structuré nécessaire avant toute décision.

Meilin, responsable du CRM de LiVrai, a mandaté cet audit pour mettre en évidence :

- Les fonctionnalités disponibles du CRM et les types d'utilisateurs concernés
- La conception technique de l'application
- Les principaux points forts de la solution
- Ses limites au regard de la croissance, de la volumétrie et des besoins de disponibilité

2. Vue fonctionnelle

Le CRM LiVrai est accessible via une page de connexion. L'application propose deux parcours distincts selon le rôle de l'utilisateur : Admin ou Client.

2.1 Parcours Admin

Le menu admin donne accès à deux sections principales : Clients et Livraisons.

Clients

- Affichage de l'ensemble des clients enregistrés dans le CRM
- Création de nouveaux clients via un formulaire

Livraisons

L'écran livraisons affiche les livraisons à venir et les livraisons passées.

Pour les livraisons à venir, l'admin peut :

- Accepter une livraison : un formulaire apparaît pour saisir le montant à facturer
- Refuser une livraison : fonctionnalité non opérationnelle (voir section 3)

Une fois facturée, la livraison passe au statut « Terminée » et apparaît dans les livraisons passées.



Diagramme de séquence Admin

2.2 Parcours Client

Le menu client donne accès à deux sections : Commande et Livraisons.

Commande

- Création d'une nouvelle demande de livraison via un formulaire (volume + poids)
- La commande créée apparaît immédiatement dans l'onglet Livraisons

Livraisons

- Affichage des livraisons à venir avec leur statut
- Affichage des livraisons passées avec le prix et le statut final

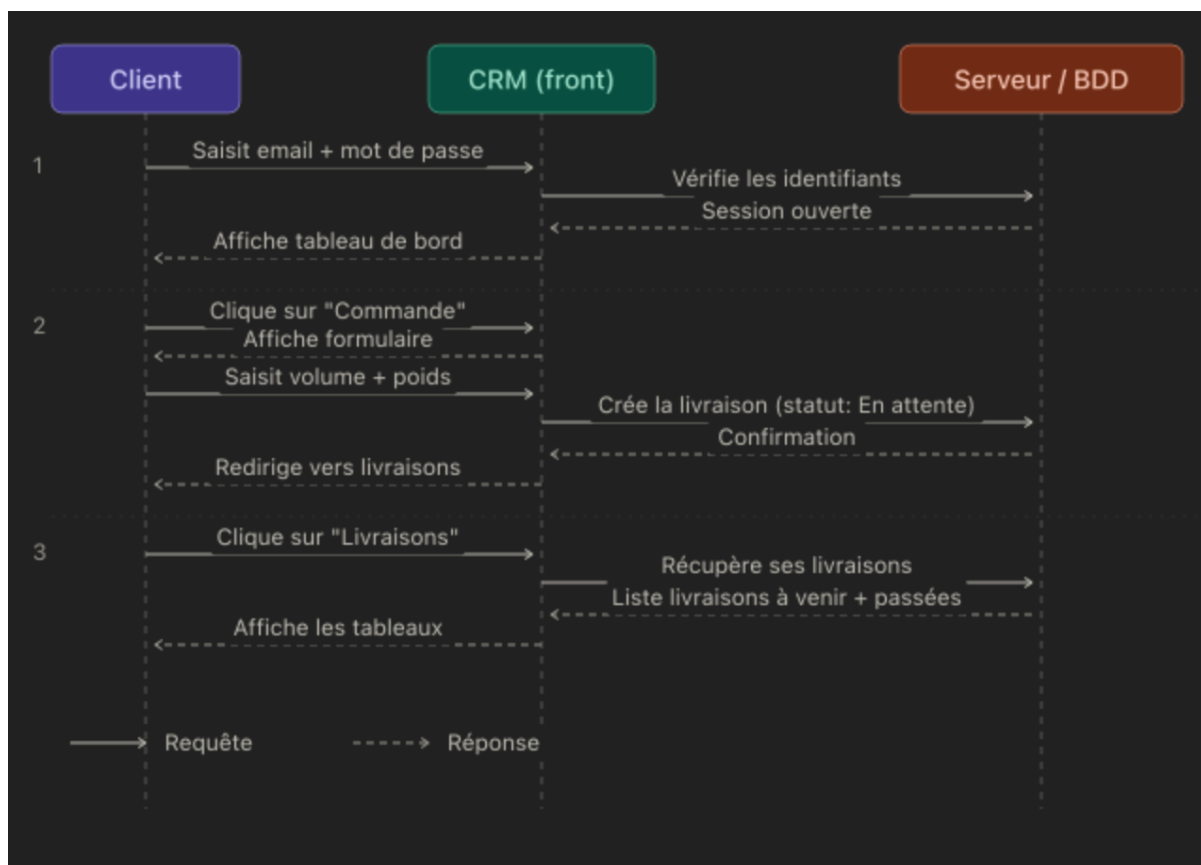


Diagramme de séquence Client

2.3 Diagramme de cas d'utilisation

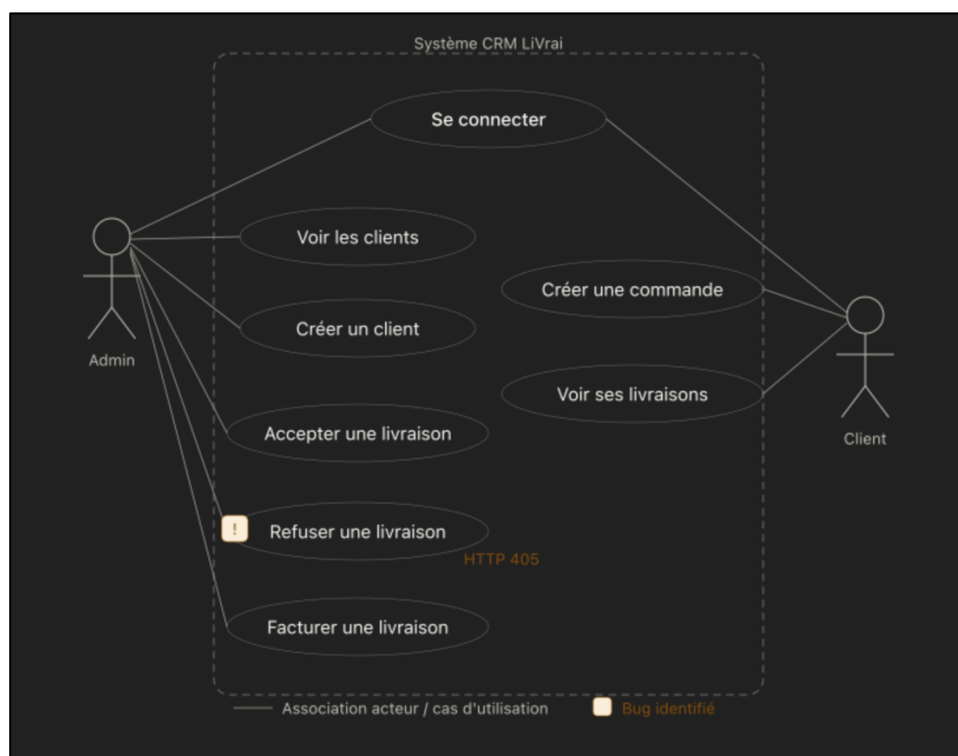


Diagramme de cas d'utilisation

2.4 Diagramme d'activité

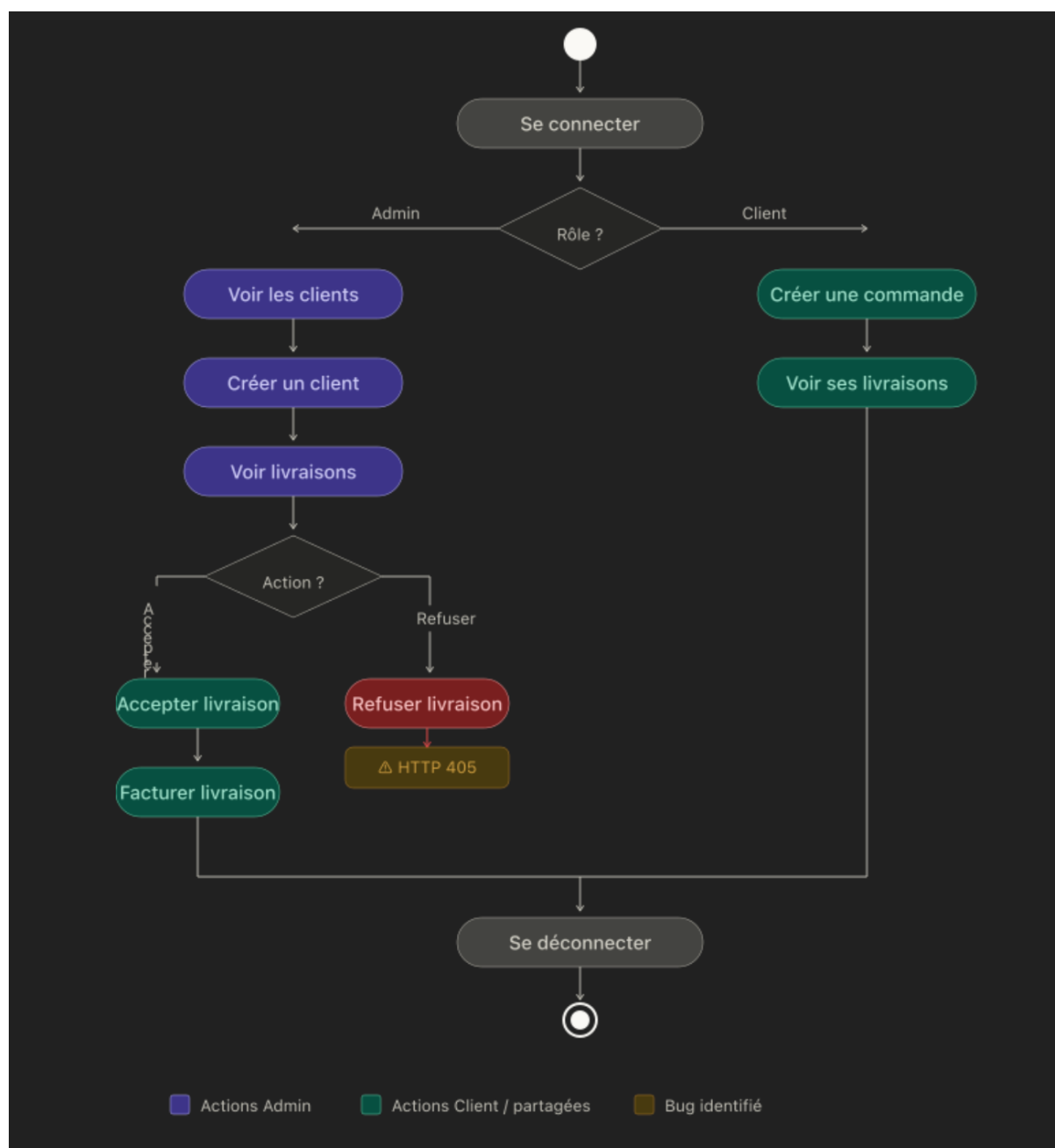


Diagramme d'activité

3. Expérience utilisateur

3.1 Anomalies d'affichage

Les tableaux de livraisons présentent des incohérences visuelles :

- Les colonnes Poids et Prix semblent fusionnées (affichage : 120 120.0) dans la vue des livraisons passées, côté admin et client.
- Les en-têtes de colonnes ne correspondent pas toujours aux données affichées.

3.2 Points de friction identifiés

Connexion

Lorsque des identifiants incorrects sont saisis, aucune alerte ni message d'erreur n'est affiché. La page effectue un simple rafraîchissement sans retour visuel pour l'utilisateur.

Facturation

Aucune demande de confirmation n'est présentée avant de valider une facturation. En cas d'erreur de saisie, l'action se déclenche immédiatement sans possibilité de revenir en arrière ou d'annuler.

Fonctionnalité de refus - bug bloquant

L'action Refuser une livraison provoque une erreur HTTP 405 (méthode POST non supportée). La fonctionnalité est présente dans l'interface mais non opérationnelle.

Cause identifiée : faute de frappe dans deliveries.jsp, le formulaire post sur /livraisons au lieu de /livraison, ciblant une servlet qui ne gère pas les requêtes POST.

⚠ Un admin ne peut pas refuser une livraison. Celle-ci reste bloquée indéfiniment au statut « En attente » côté client. Ce bug révèle une absence de tests sur cette fonctionnalité.

4. Description technique

4.1 Architecture générale

L'application suit globalement le pattern MVC (Model-View-Controller) :

Couche	Rôle	Technologies
Modèle (Model)	Objets métier	Beans Java (User, Delivery)
Contrôleur (Controller)	Traitement des requêtes HTTP	Servlets Java EE
Vue (View)	Affichage HTML	JSP + JSTL + EL
Accès données	Requêtes SQL	DAO + JDBC + MySQL 8

4.2 Authentification

Un filtre AuthenticationFilter est appliqué sur /* toutes les requêtes passent par ce filtre avant d'atteindre une servlet. Deux routes sont publiques : /login et /logout.

4.3 Couche DAO

AbstractDao

Configure la connexion à la base via JDBC (driver com.mysql.cj.jdbc.Driver). Maintient une connexion par instance de DAO via un pattern singleton.

DeliveryDao

Implémente 8 fonctions métier : createDelivery, getDeliveryById, getAllDeliveries, getAllDeliveriesByUserId, acceptDeliveryById, rejectDeliveryById, billDeliveryById, deleteDelivery. Toutes utilisent des requêtes SQL préparées (PreparedStatement), protégeant contre les injections SQL.

UserDao

Implémente 5 fonctions : addUser, getUserById, getUserByEmail, getAllNonAdminUsers, updateUser.

Problématiques identifiées

⚠ Les mots de passe sont stockés et comparés en clair, aucun hashage (bcrypt, SHA-256...) n'est utilisé. Une compromission de la base expose directement tous les credentials.

⚠ Les objets User retournés incluent systématiquement le mot de passe, même quand il n'est pas nécessaire. Dans une architecture API REST, ces données seraient exposées dans les réponses HTTP.

4.4 Beans

Les beans reflètent fidèlement la structure des tables en base de données. Cependant, le bean Delivery contient un champ clientName absent de la table delivery. Ce champ est alimenté à la volée par la servlet pour l'affichage admin, un mélange entre données métier et données de présentation.

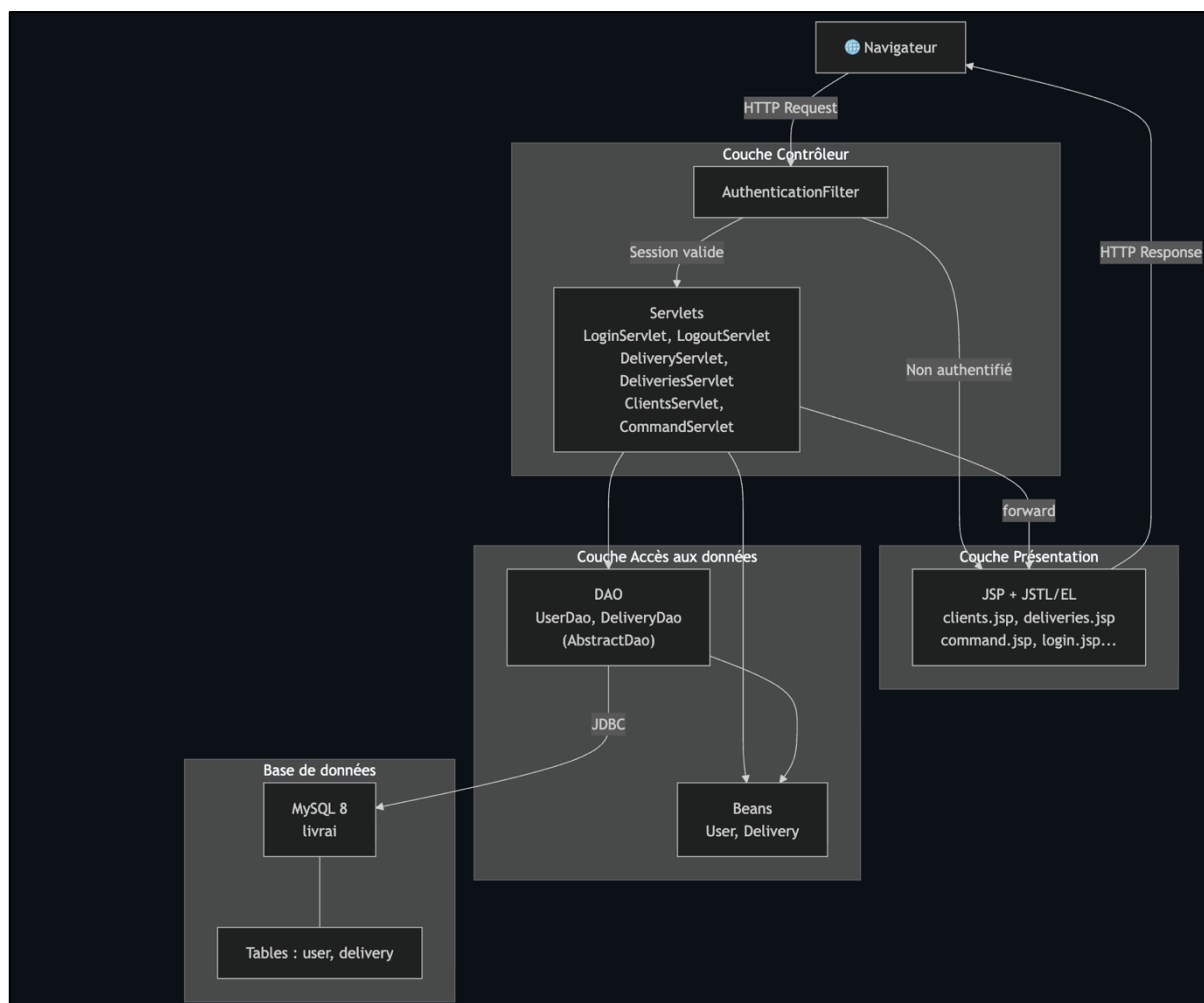
4.5 Servlets

Servlet	URL	Rôle
HomeServlet	/	Affichage de la page d'accueil
LoginServlet	/login	Gestion de la connexion (GET + POST)
LogoutServlet	/logout	Déconnexion et invalidation de session
ClientsServlet	/clients	Liste et création des clients
DeliveriesServlet	/livraisons	Affichage des livraisons (GET uniquement)
DeliveryServlet	/livraison	Actions sur une livraison (ACCEPT / REJECT / BILL)
CommandServlet	/commande	Création d'une commande client

4.6 JSP (Views)

Les vues sont toutes situées dans WEB-INF/, les rendant inaccessibles directement par URL, bonne pratique de sécurité. Elles utilisent JSTL pour la logique (boucles, conditions) et EL (`${ }`) pour afficher les données transmises par les servlets.

4.7 Diagramme de composants



5. Points forts et déficiences

5.1 Points forts

- Les requêtes SQL préparées (PreparedStatement) protègent contre les injections SQL.
- Les JSP placées dans WEB-INF/ sont inaccessibles directement par URL.
- Le filtre AuthenticationFilter sur /* assure une protection globale des routes sans dupliquer la logique.
- L'architecture suit globalement le pattern MVC, apportant une séparation de base entre les couches.

5.2 Déficiences et risques

Sécurité

- Mots de passe stockés et comparés en clair, aucun hashage utilisé.
- Les objets User retournés par les DAO incluent systématiquement le mot de passe, même inutilement.
- Aucune validation des données saisies dans les formulaires.

Architecture

- MVC incomplet : les servlets embarquent directement la logique métier (filtrage, gestion des rôles) qui devrait être déportée dans une couche service dédiée.
- Mélange entre données métier et données d'affichage dans les beans (clientName dans Delivery).
- Aucune gestion des erreurs dans les DAO et les servlets, les erreurs remontent comme RuntimeException non gérées.

Performance, disponibilité et volumétrie

- Absence de pool de connexions, chaque instance de DAO crée sa propre connexion, multipliant les connexions ouvertes sous forte charge.
- Les requêtes getAllDeliveries() ramènent l'intégralité des données sans pagination, ingérable avec la croissance des volumes.
- Aucun index défini explicitement sur la table delivery, les recherches par userId ou status seront de plus en plus lentes.
- Architecture monolithique JSP/Servlet difficile à scaler horizontalement pour absorber la croissance.

6. Conclusion

Le CRM de LiVrai remplit ses fonctions de base et présente une architecture reconnaissable. Il couvre les besoins essentiels : gestion des clients, suivi des livraisons, gestion des rôles. Certaines bonnes pratiques sont en place (requêtes préparées, protection des routes, JSP dans WEB-INF).

Cependant, l'audit révèle des limites structurelles importantes qui deviennent critiques au regard de la croissance de LiVrai :

- Des failles de sécurité significatives (mots de passe en clair, absence de validation).
- Une architecture qui mélange les responsabilités, rendant la maintenance difficile.
- Des problèmes de performance prévisibles face à la montée en charge (absence de pool de connexions, absence de pagination, absence d'index).
- Une fonctionnalité bloquante non testée (refus de livraison, HTTP 405).

Ces constats justifient une refonte technique complète. La nouvelle architecture devra répondre aux enjeux de performance, disponibilité, maintenabilité et évolutivité identifiés tout au long de cet audit.